

MECHATRONICS AND IOT

ASSIGNMENT REPORT – 10

TITLE:

Design and develop a Smart Irrigation and Water Management System. The system automates irrigation control based on environmental conditions to optimize water usage in agricultural applications.

TEAM MEMBERS:

HARISANKARAN S

(24MER013)

MUTHUKUMAR M

(24MER034)

MONESHWARAN C

(24MER032)

PROJECT OVERVIEW

The Smart Irrigation and Water Management System is designed to automate irrigation control based on environmental conditions to optimize water usage in agricultural applications. The system monitors soil moisture, temperature, and humidity using sensors. The soil-moisture sensor determines whether the soil is dry or wet. Based on the measured soil condition, the controller automatically operates the water pump through a relay. When the soil is dry, the system activates irrigation by switching the pump ON. When the soil has sufficient moisture, the pump is switched OFF. The system also sends sensor and irrigation data to Adafruit IO for remote monitoring and displays the status on an OLED display.

PROBLEM STATEMENT

- Manual irrigation of agricultural fields.
- Unnecessary water consumption.
- Difficulty in continuously monitoring soil moisture.
- Irrigation even when sufficient soil moisture is available.
- Lack of automatic pump control.
- Difficulty in monitoring environmental conditions.
- Lack of remote monitoring of irrigation status.
- Need for efficient water management in agriculture.

PROPOSED SOLUTION

The proposed system uses a **soil-moisture sensor, DHT11 sensor, microcontroller, relay, water pump, OLED display, and IoT platform** to automate irrigation. The soil-moisture sensor measures the soil condition. The DHT11 measures **temperature and humidity**. The microcontroller compares the soil-moisture value with a predefined threshold.

The system automatically:

- Detects dry and wet soil.
- Controls the water pump.
- Monitors temperature.
- Monitors humidity.
- Displays sensor values on an OLED.
- Sends data to Adafruit IO.
- Displays pump status.
- Displays irrigation status.

WORKING PRINCIPLE

- The soil-moisture sensor measures soil moisture.
- The DHT11 measures temperature and humidity.
- The microcontroller reads the sensor values.
- The soil-moisture value is compared with the predefined threshold.
- If the soil value is greater than the threshold, the soil is considered **DRY**.
- The irrigation system is activated and the pump turns **ON**.
- If the soil value is below the threshold, the soil is considered **WET**.
- Irrigation is stopped and the pump turns **OFF**.
- Temperature, humidity, soil moisture, pump status, and irrigation status are sent to Adafruit IO.
- The current values are displayed on the OLED.
- The process continuously repeats.

SYSTEM ARCHITECTURE

Soil Moisture Sensor + DHT11 → ESP32/Microcontroller → Irrigation Control → Relay
→ Water Pump

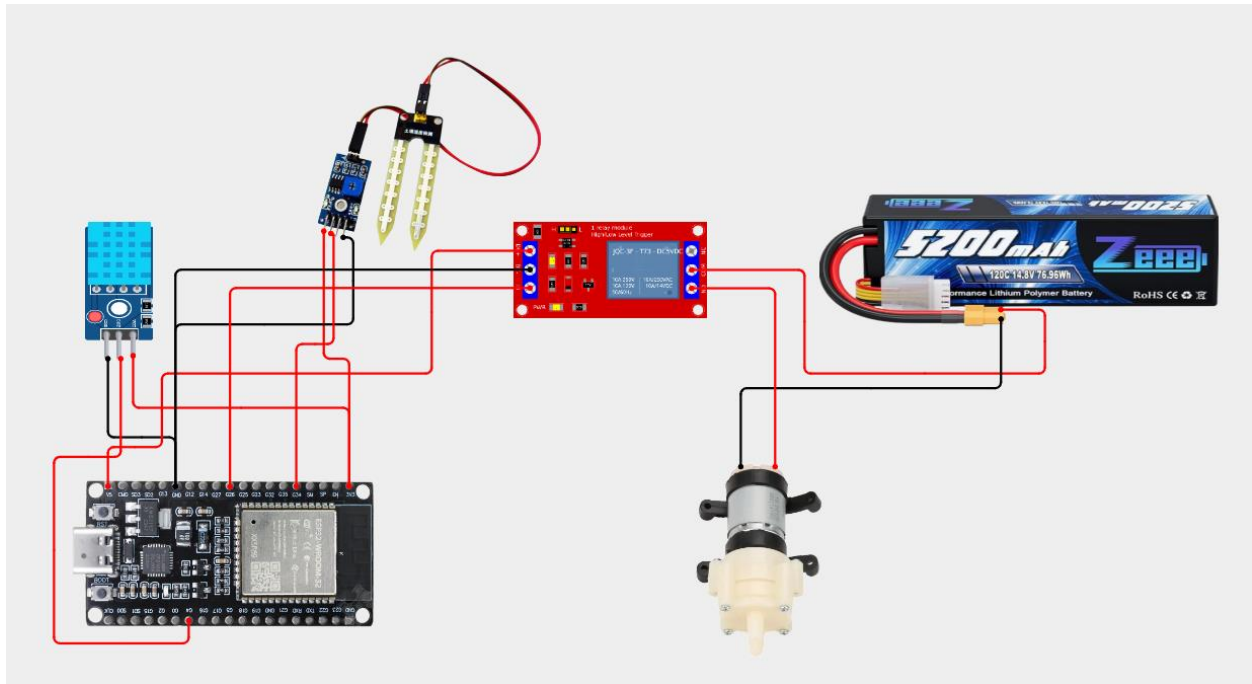
IRRIGATION OPERATING CONDITIONS

Parameter	Normal	Warning	Critical
Soil Moisture	Wet/Sufficient	Moisture decreasing	Dry
Pump Status	OFF	—	ON
Irrigation Status	Not Required	Monitoring	Required
Temperature	Normal	High	Very High
Humidity	Normal	Low/High	Abnormal
Soil Condition	Wet	Drying	Very Dry

PARAMETERS MONITORED

Parameter	Sensor/Device	Purpose
Soil Moisture	Soil Moisture Sensor	Determine soil condition
Temperature	DHT11	Measure temperature
Humidity	DHT11	Measure humidity
Pump Status	Relay	Control/indicate pump
Irrigation Status	Microcontroller	Indicate whether irrigation is required
System Status	OLED	Display current condition
Cloud Data	Adafruit IO	Remote monitoring

CIRCUIT DESIGN



ALGORITHM

1. Initialize the microcontroller.
2. Initialize the DHT11 sensor.
3. Initialize the soil-moisture sensor.
4. Initialize the relay.
5. Initialize the OLED display.
6. Connect to Adafruit IO.
7. Read temperature and humidity.
8. Read soil-moisture value.
9. Compare the soil-moisture value with the threshold.
10. If soil value is greater than the threshold:
 - Set irrigation status to **ON**.
 - Set pump status to **ON**.
 - Turn ON the relay/pump.
11. Otherwise:
 - Set irrigation status to **OFF**.
 - Set pump status to **OFF**.
 - Turn OFF the relay/pump.

12. Send sensor data to Adafruit IO.
13. Display temperature, humidity, soil moisture, pump status, and irrigation status on the OLED.
14. Continue monitoring.
15. **REPEAT.**

CODING

```
#include "AdafruitIO_WiFi.h"
```

```
#include <Wire.h>
```

```
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h>
```

```
#include <DHT.h>
```

```
// =====
```

```
// WIFI DETAILS
```

```
// =====
```

```
#define WIFI_SSID    "OnePlus Nord CE5 7C5C"
```

```
#define WIFI_PASSWORD "qkts5737"
```

```
// =====
```

```
// ADAFRUIT IO DETAILS
```

```
// =====
```

```
#define IO_USERNAME  "Hair_13"
```

```
#define IO_KEY       "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"
```

```
// Create Adafruit IO WiFi object
```

```
AdafruitIO_WiFi io(
```

```
  IO_USERNAME,
```

```
  IO_KEY,
```

```
  WIFI_SSID,
```

```
  WIFI_PASSWORD
```

```
);
```

```
// =====  
// FEEDS  
// =====  
  
AdafruitIO_Feed *soilFeed =  
  io.feed("soil-moisture");  
  
AdafruitIO_Feed *temperatureFeed =  
  io.feed("temperature");  
  
AdafruitIO_Feed *humidityFeed =  
  io.feed("humidity");  
  
AdafruitIO_Feed *pumpFeed =  
  io.feed("pump-status");  
  
AdafruitIO_Feed *irrigationFeed =  
  io.feed("irrigation-status");  
  
// =====  
// DHT11  
// =====  
  
#define DHT_PIN 4  
#define DHT_TYPE DHT11  
  
DHT dht(DHT_PIN, DHT_TYPE);  
  
// =====  
// SOIL MOISTURE SENSOR  
// =====  
  
#define SOIL_PIN 34  
  
// =====  
// RELAY
```

```
// =====

#define RELAY_PIN 26

// =====
// OLED
// =====

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);

// =====
// VARIABLES
// =====

float temperature = 0;
float humidity = 0;

int soilValue = 0;

bool pumpStatus = false;

bool irrigationStatus = false;

// =====
// SOIL MOISTURE THRESHOLD
// =====

// Adjust this value after testing your soil sensor
```

```
int soilThreshold = 2000;
```

```
// =====  
// TIMING  
// =====
```

```
unsigned long lastSend = 0;
```

```
const unsigned long sendInterval =  
    15000;
```

```
// =====  
// SETUP  
// =====
```

```
void setup()  
{
```

```
    Serial.begin(9600);
```

```
    delay(1000);
```

```
// -----  
// DHT  
// -----
```

```
dht.begin();
```

```
// -----  
// SENSOR PINS  
// -----
```

```
pinMode(SOIL_PIN, INPUT);
```

```
pinMode(RELAY_PIN, OUTPUT);
```

```
// -----  
// RELAY OFF AT STARTUP  
// -----
```

```
// Most relay modules are ACTIVE LOW
```

```
digitalWrite(  
    RELAY_PIN,  
    HIGH  
);
```

```
// -----  
// OLED  
// -----
```

```
Wire.begin(21, 22);
```

```
if (!display.begin(  
    SSD1306_SWITCHCAPVCC,  
    0x3C))  
{
```

```
    Serial.println(  
        "OLED not detected!"  
    );
```

```
}  
else  
{
```

```
    display.clearDisplay();
```

```
    display.setTextColor(  
        SSD1306_WHITE  
    );
```

```
    display.setTextSize(1);
```

```
display.setCursor(10, 10);

display.println(
  "SMART IRRIGATION"
);

display.setCursor(10, 30);

display.println(
  "Starting..."
);

display.display();
}

// -----
// CONNECT TO ADAFRUIT IO
// -----

Serial.println();

Serial.println(
  "Connecting to Adafruit IO..."
);

io.connect();

// Wait for connection

while (io.status() < AIO_CONNECTED)
{

  Serial.print(".");

  delay(500);
}
```

```
Serial.println();

Serial.println(
  "Connected to Adafruit IO!"
);

display.clearDisplay();

display.setTextSize(1);

display.setCursor(0, 10);

display.println(
  "Adafruit IO"
);

display.setCursor(0, 30);

display.println(
  "Connected!"
);

display.display();

delay(2000);
}

// =====
// LOOP
// =====

void loop()
{

  // Keep Adafruit IO connection alive
```

```
io.run();

// -----
// SEND SENSOR DATA
// -----

if (
  millis() - lastSend >=
  sendInterval
)
{

  lastSend = millis();

  // =====
  // DHT11
  // =====

  temperature =
    dht.readTemperature();

  humidity =
    dht.readHumidity();

  if (isnan(temperature) ||
      isnan(humidity))
  {

    Serial.println(
      "DHT11 reading failed!"
    );

  }
  else
  {
```

```
Serial.print(
  "Temperature: "
);

Serial.print(
  temperature
);

Serial.println(" C");


Serial.print(
  "Humidity: "
);

Serial.print(
  humidity
);

Serial.println(" %");
}


// =====
// SOIL MOISTURE SENSOR
// =====

soilValue =
  analogRead(SOIL_PIN);

Serial.print(
  "Soil Moisture: "
);

Serial.println(
  soilValue
);
```

```

// =====
// IRRIGATION CONTROL
// =====

// Higher value = dry soil
// Lower value = wet soil
//
// Adjust soilThreshold according
// to your sensor calibration.

if (soilValue > soilThreshold)
{

    irrigationStatus = true;

    pumpStatus = true;

    Serial.println(
        "Soil is DRY"
    );

    Serial.println(
        "Irrigation Required"
    );

    Serial.println(
        "Pump ON"
    );

    // ACTIVE LOW RELAY

    digitalWrite(
        RELAY_PIN,
        LOW
    );

}

```

```
else
{

    irrigationStatus = false;

    pumpStatus = false;

    Serial.println(
        "Soil is WET"
    );

    Serial.println(
        "Irrigation Not Required"
    );

    Serial.println(
        "Pump OFF"
    );

    // ACTIVE LOW RELAY

    digitalWrite(
        RELAY_PIN,
        HIGH
    );
}

// =====
// SEND DATA TO ADAFRUIT IO
// =====

Serial.println();

Serial.println(
    "Sending data to Adafruit IO..."
);
```

```
if (!isnan(temperature))  
{
```

```
    temperatureFeed->save(  
        temperature  
    );  
}
```

```
if (!isnan(humidity))  
{
```

```
    humidityFeed->save(  
        humidity  
    );  
}
```

```
soilFeed->save(  
    soilValue  
);
```

```
pumpFeed->save(  
    pumpStatus ? 1 : 0  
);
```

```
irrigationFeed->save(  
    irrigationStatus ? 1 : 0  
);
```

```
Serial.println(  
    "Data sent successfully!"  
);
```

```
// =====
```

```
// OLED DISPLAY
// =====

display.clearDisplay();

display.setTextColor(
  SSD1306_WHITE
);

display.setTextSize(1);

// Temperature and Humidity

display.setCursor(0, 0);

display.print("T:");

display.print(
  temperature,
  1
);

display.print("C ");

display.print("H:");

display.print(
  humidity,
  0
);

display.println("%");

// Soil Moisture

display.setCursor(0, 15);
```

```
display.print(  
  "Soil: "  
);  
  
display.println(  
  soilValue  
);  
  
// Pump Status  
  
display.setCursor(0, 30);  
  
display.print(  
  "Pump: "  
);  
  
if (pumpStatus)  
{  
  
  display.println(  
    "ON"  
  );  
  
}  
else  
{  
  
  display.println(  
    "OFF"  
  );  
}  
  
// Irrigation Status  
  
display.setCursor(0, 45);
```

```

if (irrigationStatus)
{

    display.println(
        "IRRIGATION ON"
    );

}
else
{

    display.println(
        "SOIL OK"
    );
}

display.display();


// =====
// SERIAL SEPARATOR
// =====

Serial.println(
    "=====
");
}
}

```

SYSTEM WORKING

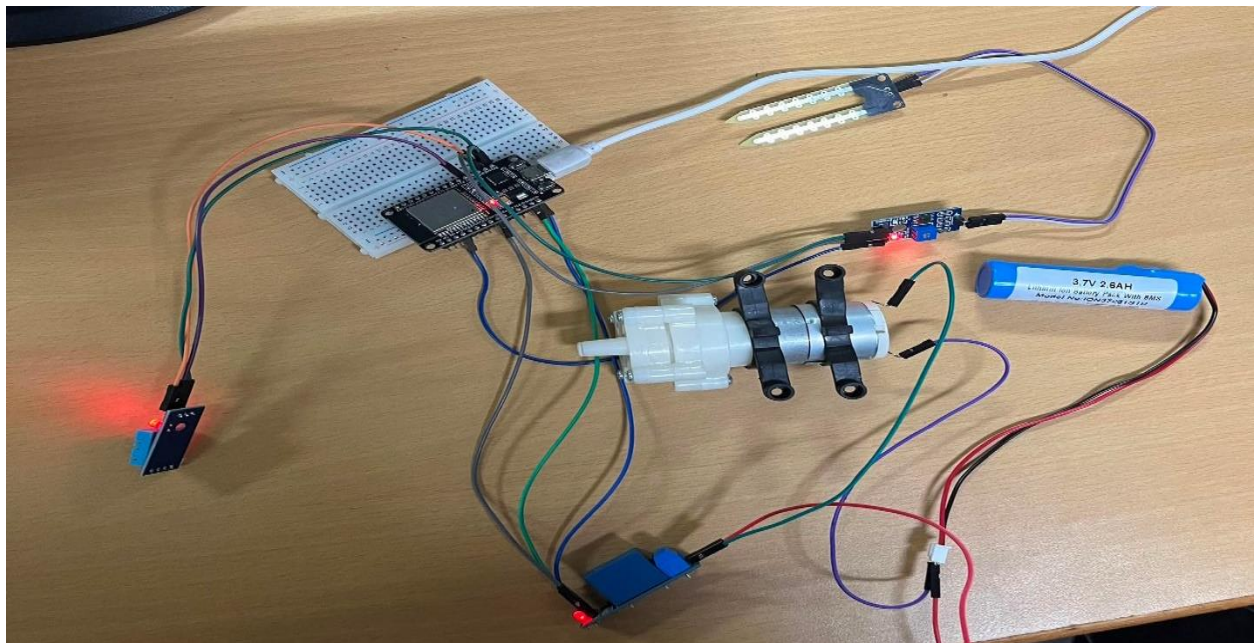
The system starts by initializing the sensors, relay, OLED display, and IoT connection. The DHT11 sensor measures temperature and humidity, while the soil-moisture sensor measures the soil condition. The soil-moisture value is compared with the predefined threshold. In the given implementation, a value greater than 2000 indicates dry soil, while a lower value indicates wet soil. When the soil is dry, the controller sets the irrigation status and pump status to ON and activates the relay. This starts the irrigation pump. When the soil becomes sufficiently wet, the controller switches the relay OFF and stops irrigation. The sensor readings and irrigation information are then uploaded to Adafruit IO, while the OLED displays the current temperature, humidity, soil value, pump status, and irrigation status.

COMPONENTS

S.No.	Component	Quantity	Purpose
1	ESP32/Microcontroller	1	Main controller
2	DHT11 Sensor	1	Temperature and humidity measurement
3	Soil Moisture Sensor	1	Soil moisture measurement
4	Relay Module	1	Pump switching
5	Water Pump	1	Irrigation
6	OLED Display	1	Display system information
7	Wi-Fi Connection	1	IoT communication
8	Power Supply	1	Power source
9	Breadboard & Jumper Wires	1 set	Circuit connections

PROTOTYPE IMPLEMENTATION

The prototype can be implemented using an ESP32/microcontroller, DHT11 sensor, soil-moisture sensor, relay, water pump, OLED display, and Wi-Fi/Adafruit IO platform. The soil-moisture sensor is placed in the agricultural soil to measure its moisture condition. The DHT11 sensor measures the surrounding temperature and humidity. The sensors are connected to the microcontroller. The controller processes the sensor readings and determines whether irrigation is required. The relay is used to control the water pump. When the soil is dry, the relay activates the pump. When sufficient moisture is detected, the relay switches the pump OFF. The OLED provides local information, while Adafruit IO provides remote monitoring of the sensor and irrigation data.



RESULTS & PERFORMANCE ANALYSIS

Condition	Soil Condition	Pump	Irrigation
Soil sufficiently wet	WET	OFF	Not Required
Soil becomes dry	DRY	ON	Required
Soil remains dry	DRY	ON	Continued
Soil becomes wet	WET	OFF	Stopped

PERFORMANCE

- Automatic soil-moisture monitoring
- Automatic irrigation control
- Automatic water-pump control
- Reduced unnecessary water usage
- Temperature monitoring
- Humidity monitoring
- Real-time soil-moisture monitoring
- OLED-based local display
- IoT-based remote monitoring
- Pump-status monitoring
- Irrigation-status monitoring
- Reduced dependence on manual irrigation
- Improved water-management efficiency
- Low-cost prototype suitable for agricultural and educational applications

